

VII IEEE Floating Point

Contreras R Orlando^{1,2*} and Garcia B Iker^{1,2*†}

³Departamento de Sistemas Electrónicos, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): {al348390,al307630}@edu.uaa.mx;

†Ambos autores contribuyeron de forma equitativa a este trabajo.

Abstract

Este documento tiene como objetivo implementar un algoritmo de conversión del formato decimal a punto flotante para valores de 32 bits este formato ofrece una gran precisión y es de los más usados actualmente para operaciones aritméticas.

Keywords: VHDL, FPGA, IEEE, Fraccionario, Punto Flotante

1 Introducción

Este convertidor se destaca por transformar números decimales en su representación en punto flotante según el estándar IEEE 754, utilizando técnicas que permiten manejar con precisión tanto la normalización del número como la codificación del signo, el exponente y la mantisa.

El procesamiento eficiente de señales digitales y aplicaciones de alto rendimiento requiere conversiones numéricas rápidas y precisas. Entre ellas, la conversión de números decimales a formatos de punto flotante es esencial, ya que este tipo de representación es ampliamente utilizado en algoritmos científicos, gráficos por computadora, inteligencia artificial y simulaciones numéricas.

En este contexto, el uso de circuitos diseñados en {FPGA} ofrece ventajas notables en términos de velocidad, paralelismo y eficiencia energética, comparado con soluciones basadas en software.

2 Metodología

2.1 CLZ Count Leading Zeros

Este módulo busca replicar el comportamiento de la instrucción CLZ, dicha instrucción consiste contar el número de ceros desde la parte izquierda, curiosamente este método es una aproximación al logaritmo base 2 es decir

$$32 - CLZ(x) \approx \log_2(x)$$

Esto es muy útil debido a que nos permite saber el exponente de un número, así mismo nos permite saber la cantidad de ceros que deberemos de recorrer para eliminar el primer 1 según el algoritmo de IEEE. Se implementó como componente debido al uso que se le da en la arquitectura ya que sirve tanto para el valor entero como el valor fraccionario. Así mismo debe destacarse que se realiza de forma combinacional, siendo un simple decodificador de prioridad de 32 casos.

2.2 b2f Binary To Fract

Por otra parte, este módulo presentó un poco más de complejidad; además, se realizó de forma secuencial. El algoritmo que se implementó fue prácticamente ir multiplicando por 2 el valor de entrada hasta que llegue a un valor fijo. En el momento en que exceda ese valor, se le concatenará un 1 al vector resultado. Es importante mencionar que el vector resultado, en cada iteración, se estará desplazando a la izquierda.

El valor fijo del que se habla será la resolución que se tendrá siendo que si se ingresa el valor de 1 el valor mínimo sería

$$\frac{1}{Valor_{fijo}}$$

Dicho valor se maneja como una constante por lo que para aumentar sería necesario que se ajustara en la constante.

Por otra parte, la precisión de los bits depende meramente del número de iteraciones del proceso; nosotros definimos esto como 32 bits. Para valores menores, será necesario incrementar las iteraciones. Al igual que el otro bloque, se utilizó como componente para instanciarse cada vez que se le requiera

2.3 IEEE Standart

Finalmente nuestro bloque top o nuestro componente principal se reduce a meras concatenaciones y multiplexores para la Mantisa, se observa que el bit de signo sale como entra sin modificar el número, Para la parte del exponente se calcula por medio del bloque CLZ mientras que para la Mantisa se utiliza una serie de shifteos así como señales de control para detectar si se está ingresando un valor nulo (es decir un 0 tanto en la parte entera como en la fraccionaria) o si se está ingresando un número meramente fraccionario.

3 Resultados

En la simulación podemos apreciar como se convierten los valores de forma exitosa, se debe mencionar que para simularlo se instanció la entidad con múltiples valores, es por ello que se puede ver cómo se ejecutó en paralelo, se debe hacer mención de como el valor 0.001 tomo al rededor de 1000 ns, esto es algo que sin duda se debe optimizar o considerar para futuras implementacioens del módulo

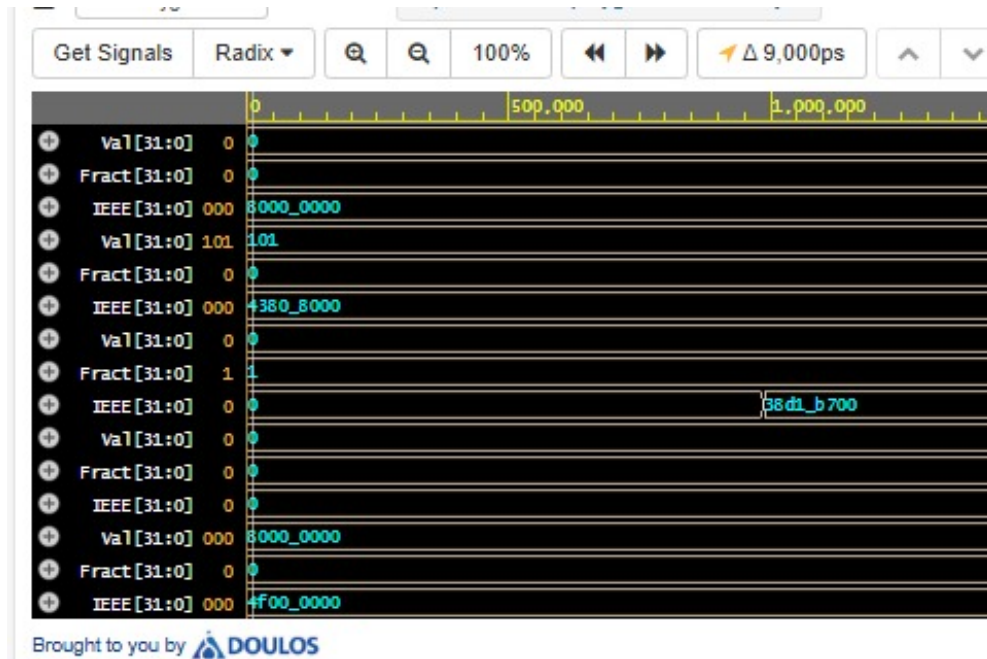


Fig. 1 Testbench realizado por EDA Playground

Device Utilization Summary					
Slice Logic Utilization	Used	Available	Utilization	Note(s)	
Number of Slice Registers	72	11,440	1%		
Number used as Flip Flops	72				
Number used as Latches	0				
Number used as Latch-thrus	0				
Number used as AND/OR logics	0				
Number of Slice LUTs	271	5,720	4%		
Number used as logic	271	5,720	4%		
Number using O6 output only	188				
Number using O5 output only	5				
Number using O5 and O6	78				
Number used as ROM	0				
Number used as Memory	0	1,440	0%		
Number of occupied Slices	89	1,430	6%		
Number of MUXCYs used	52	2,860	1%		
Number of LUT Flip Flop pairs used	271				
Number with an unused Flip Flop	202	271	74%		
Number with an unused LUT	0	271	0%		
Number of fully used LUT-FF pairs	69	271	25%		
Number of unique control sets	2				
Number of slice register sites lost to control set restrictions	8	11,440	1%		
Number of bonded IOBs	74	102	72%		
IOB Latches	8				
Number of RAMB16BWERS	0	32	0%		
Number of RAMB8BWERS	0	64	0%		
Number of BUFIO2/BUFIO2_2CLKs	0	32	0%		
Number of BUFIO2FB/BUFIO2FB_2CLKs	0	32	0%		
Number of BUFG/BUFGMUXs	1	16	6%		
Number used as BUFGs	1				
Number used as BUFGMUX	0				
Number of DCM/DCM CLKGENs	0	4	0%		

Fig. 2 Summary report generated by Xilinx

4 Conclusiones

La complejidad de esta práctica se remitió a la parte fraccionaria, debido a que en un primer momento se construyó el bloque principal funcionando para después realizar el conversor de parte fraccionaria, dicho conversor se buscaba implementar recortando la señal de forma dinámica, es decir `Result(31-CLZFrac downto 0) ZEROs(CLZ downto 0)` pero marcaba error debido a que CLZFrac podría alcanzar el valor de 32 haciendo el indexado de forma errónea, esta señal nunca lograría alcanzar ese valor debido a la restricción que tenía pero incluso así marcaba error, la opción mas fácil fue utilizar la función `shiftleft()` de la librería NUMERIC STD. De esta forma se pudo completar la práctica.

En esta ocasión no se utilizó la magnífica FPGA Tang Nano pero se debe hacer mención a la portabilidad de la herramienta web EDA PlayGround debido a la forma que permite hacer testbench de forma web de una forma sencilla sin requerir de una buena computadora permitiéndolo así servir como una buena alternativa comparada con las opciones populares.

5 Anexos

Esto es la parte más importante de los RTL, si se gusta observar con mayor detalle, los PDFs estarán en el documento adjunto. Asimismo se podrá observar el código actualizado en: [GitHub Repository](#)

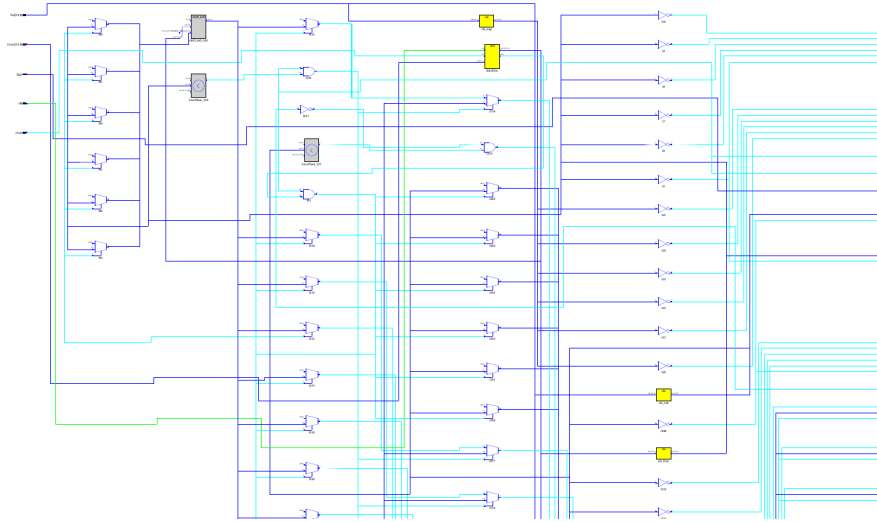


Fig. 3 RTL del Componente principal

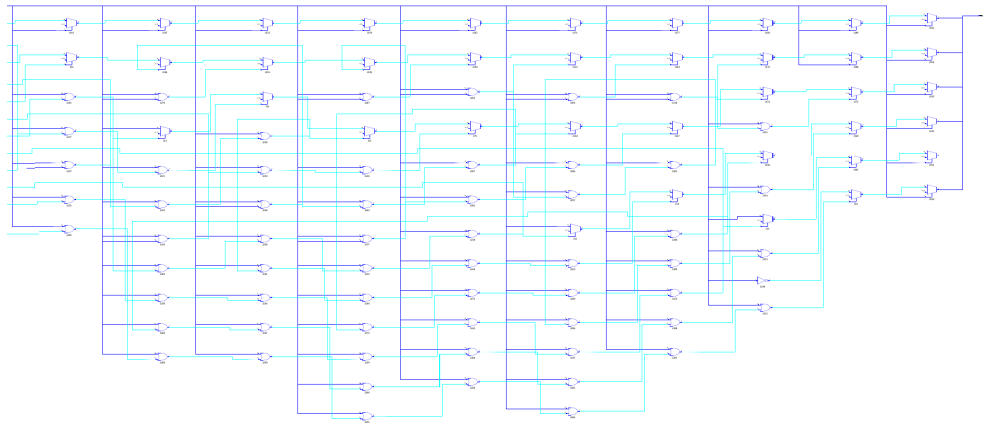


Fig. 4 RTL del Componente CLZ

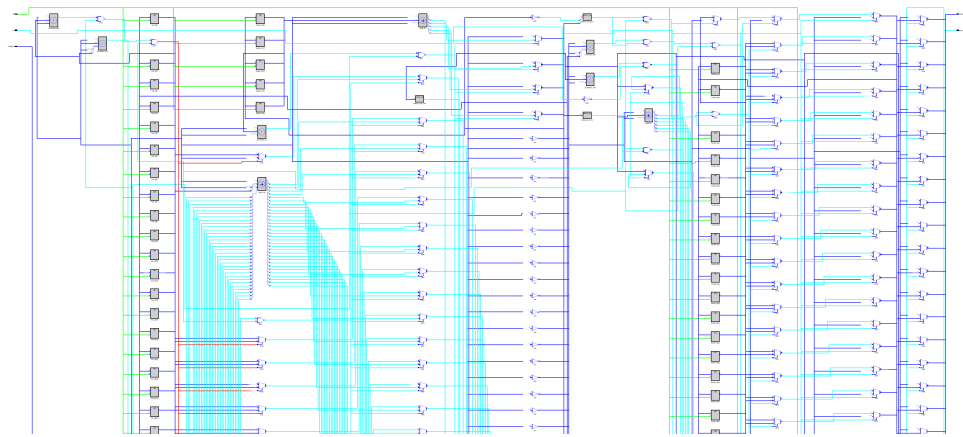


Fig. 5 RTL del Componente Binario a Fraccion